Institut National des Postes et Télécommunications (INPT)
Filière AMOA — Cours SOA

Gestion intelligente des stages et des offres de recrutement

Rapport technique du projet SOA

3 services SOAP (JAX-WS) · 3 services REST (JAX-RS) · orchestration BPMN 2.0 · interface web

Réalisé par
Prénom NOM
Prénom NOM

Encadré par
Pr. NOM DE L'ENSEIGNANT

Année universitaire 2026–2027

Table des matières

1 Introduction **1**

1.1 Contexte . . . 1
1.2 Objectifs . . . 1
1.3 Synthèse de la réalisation . . . 1
1.4 Organisation du rapport . . . 1

2 Architecture SOA **3**

2.1 Vue d'ensemble . . . 3
2.2 Principes SOA appliqués . . . 3
2.3 Choix technologiques . . . 4
2.4 Interactions entre services . . . 4
2.5 Déploiement . . . 4

3 Modèle de données **7**

3.1 Schéma relationnel . . . 7
3.2 Énumérations . . . 7
3.3 Jeu de données de démonstration . . . 7

4 Services SOAP **9**

4.1 Conception commune . . . 9
4.2 Service 1 — Gestion des étudiants . . . 9
4.3 Service 2 — Gestion des entreprises . . . 9
4.4 Service 3 — Gestion des offres de stage . . . 10
4.5 Extrait de WSDL . . . 11
4.6 Exemples d'échanges . . . 12
4.7 Implémentation . . . 13

5 Services REST **14**

5.1 Conventions communes . . . 14
5.2 Service 4 — Gestion des candidatures . . . 15
5.3 Service 5 — Notifications . . . 16
5.4 Service 6 — Suivi des stages . . . 17
5.5 API d'orchestration . . . 17

6 Sécurité des échanges **19**

6.1 Authentification . . . 19
6.2 Protection des informations sensibles . . . 19
6.3 Contrôle d'accès par rôle . . . 20

6.4 Confidentialité du transport 20

7 Orchestration BPMN **21**
7.1 Outil et principe 21
7.2 Structure du processus 21
7.3 Passerelles et conditions 21
7.4 Gestion des erreurs et des indisponibilités 21
7.5 Autres mécanismes 24
7.6 Reproduction dans Bonita Studio 25

8 Règles de gestion **26**

9 Scénarios de test **29**
9.1 Stratégie 29
9.2 Tests automatisés 29
9.3 Collection Postman 31
9.4 Requêtes SoapUI 32

10 Interface graphique (bonus) **33**
10.1 Principes 33
10.2 Parcours de l'étudiant 34
10.3 E-mails reçus 35
10.4 Parcours de l'entreprise 35
10.5 Parcours du responsable 36

11 Bilan et perspectives **38**
11.1 Bilan 38
11.2 Limites 38
11.3 Perspectives 38

A Mise en œuvre **39**
A.1 Lancement 39
A.2 Comptes de démonstration 39
A.3 Organisation du code source 39
A.4 Vidéo de démonstration 40

Table des figures

2.1 Architecture générale de la plateforme 3
2.2 Diagramme de séquence du traitement d'une candidature recevable puis acceptée . 6

3.1 Modèle de données (bleu : services SOAP, vert : services REST ; pointillés : références logiques entre domaines) 7

5.1 Automate des statuts d'une candidature (états finals en gris et vert) 15

7.1 Diagramme BPMN du processus de traitement d'une candidature (rendu bpmn-js du fichier exécuté) 22

10.1 Offres de stage : score d'adéquation et motifs d'incompatibilité calculés par `verifierCompatibilite` 34
10.2 Résultat d'une soumission de candidature 34
10.3 Notifications et suivi du stage 35
10.4 Vrais e-mails envoyés en SMTP par le service Notifications et capturés par Mailpit 35
10.5 Évaluation d'une candidature par l'entreprise : « Examiner » prend la candidature en charge, puis la décision est rendue 35
10.6 Suivi d'une instance : activités traversées en vert sur le diagramme, trace d'exécution et variables 36
10.7 Gestion d'une indisponibilité de service depuis l'interface 36
10.8 Écrans d'administration 37

Liste des tableaux

1.1 Couverture du cahier des charges . . . 2

2.1 Propriété des données par service . . . 4
2.2 Pile technique . . . 5
2.3 Matrice des appels inter-services . . . 5
2.4 Points d'accès publiés . . . 6

3.1 Valeurs énumérées . . . 8

4.1 Codes de la faute métier `ServiceFault` . . . 9
4.2 Opérations du service Étudiants . . . 10
4.3 Opérations du service Entreprises . . . 10
4.4 Opérations du service Offres . . . 11

5.1 Codes HTTP retournés . . . 14
5.2 Points d'accès du service Candidatures . . . 15
5.3 Points d'accès du service Notifications . . . 16
5.4 Points d'accès du service Suivi des stages . . . 17
5.5 Points d'accès de l'orchestration . . . 18

6.1 Matrice des droits (L : lecture, E : écriture, P : périmètre propre uniquement) . . . 20

7.1 Correspondance entre les étapes du sujet et le modèle BPMN . . . 23
7.2 Passerelles exclusives et expressions de condition . . . 23

8.1 Règles de gestion . . . 26

9.1 Scénarios de test automatisés et résultats . . . 29
9.2 Requêtes SOAP fournies . . . 32

10.1 Écrans par acteur . . . 33

A.1 Comptes de démonstration . . . 39

CHAPITRE 1

Introduction

1.1 Contexte

Dans le cadre de la transformation numérique des établissements d'enseignement supérieur, l'université souhaite disposer d'une plateforme gérant l'ensemble du cycle de vie d'un stage : publication des offres par les entreprises partenaires, candidatures des étudiants, sélection, puis suivi du stage jusqu'à son évaluation finale.

Ces activités mobilisent des acteurs et des référentiels distincts (scolarité, relations entreprises, bureau des stages) qui évoluent à des rythmes différents. Une **architecture orientée services** (SOA) est donc pertinente : chaque domaine métier est exposé sous la forme d'un service autonome, au contrat explicite, et un **processus métier BPMN** orchestre ces services pour automatiser le traitement d'une candidature, depuis son dépôt jusqu'à la décision finale.

1.2 Objectifs

Le projet répond aux exigences suivantes du cahier des charges :

— trois services **SOAP** décrits en WSDL : gestion des étudiants, des entreprises et des offres de stage ;

— trois services **REST** : gestion des candidatures, notifications, suivi des stages ;

— un **processus BPMN** d'orchestration comportant tâches de service, événements de début et de fin, passerelles de décision et gestion des erreurs ou indisponibilités de services ;

— la validation des données, la gestion des erreurs et la protection des échanges ;

— en bonus, une **interface graphique** pour les étudiants, les entreprises et les responsables.

1.3 Synthèse de la réalisation

1.4 Organisation du rapport

Le chapitre 2 présente l'architecture SOA et les interactions entre services ; le chapitre 3 le modèle de données. Les chapitres 4 et 5 documentent les opérations SOAP et les API REST, le chapitre 6 la sécurité. Le chapitre 7 détaille le processus BPMN et la gestion des erreurs, le chapitre 8 les règles de gestion, le chapitre 9 les scénarios de test et le chapitre 10 l'interface graphique.

Table 1.1 – Couverture du cahier des charges

Exigence	**Réalisation**	**Chapitre**
3 services SOAP + WSDL	JAX-WS (Metro), 24 opérations, WSDL publiés sous `/ws/*?wsdl`	4
3 services REST	JAX-RS (Jersey), 23 points d'accès JSON (+ 7 pour l'orchestration), codes HTTP normalisés	5
Persistance	JDBC sur H2 (défaut) ou PostgreSQL, testé sur les deux	3
Processus BPMN	BPMN 2.0 exécuté par Flowable, 4 couloirs, 3 sous-processus, 6 passerelles	7
Erreurs et indisponibilités	Reprise exponentielle, événements d'erreur, tâche d'incident, simulateur de pannes	7.4
Sécurité	WS-Security (SOAP), clé d'API ou HTTP Basic (REST), PBKDF2, masquage des données personnelles, contrôle par rôle	6
Notifications	Vrais e-mails HTML envoyés en SMTP et capturés par la boîte de test Mailpit	5
Règles de gestion	32 règles formalisées et implémentées	8
Tests	28 tests d'intégration JUnit, collection Postman (45 assertions), requêtes SoapUI	9
Bonus Front-End	Application web à trois espaces aux couleurs de l'INPT, visualisation BPMN en direct	10

CHAPITRE 2

Architecture SOA

2.1 Vue d'ensemble

La plateforme est découpée en six services métier, une couche d'orchestration et une interface web (figure 2.1). Tous sont hébergés par un même serveur HTTP pour simplifier le déploiement, mais chaque service reste **autonome** : il possède son contrat (WSDL ou API REST), ses propres tables et ses règles de sécurité, et **n'accède jamais directement aux données d'un autre service** ; toute interaction passe par un appel réseau SOAP ou REST. Chaque service pourrait ainsi être déployé séparément sans modification du code, simplement en changeant l'adresse de base.

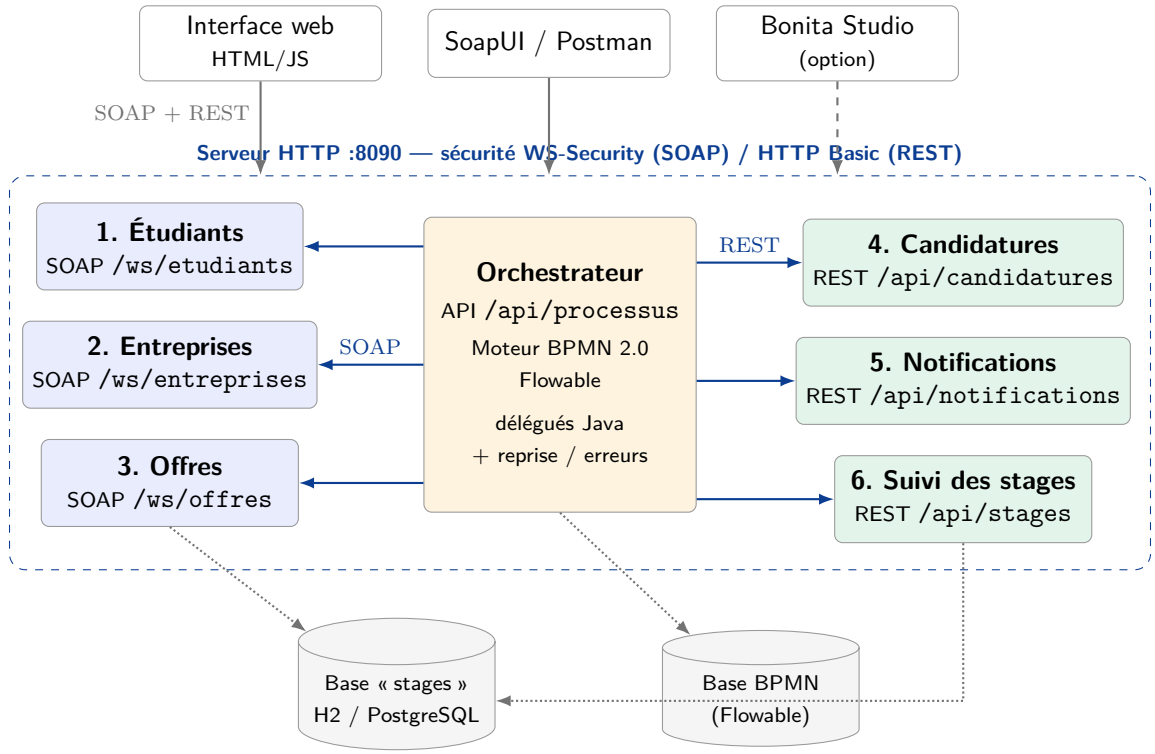


FIGURE 2.1 – Architecture générale de la plateforme

2.2 Principes SOA appliqués

Contrat standardisé

Les services SOAP sont décrits par un WSDL 1.1 (style *document/literal*) et des schémas XSD générés à partir des classes JAXB. Les services REST suivent une convention uniforme : ressources nommées au pluriel, JSON, codes HTTP normalisés et corps d'erreur commun.

Autonomie et propriété des données

Chaque service est seul propriétaire de ses tables (tableau 2.1). Lorsqu'un service a besoin d'une donnée d'un autre domaine, il l'obtient par un appel de service.

Couplage faible
L'orchestrateur ne connaît que les contrats : les clients SOAP sont créés dynamiquement à partir du WSDL publié (`Service.create(url?wsdl)`) et les appels REST passent par HTTP.

Composition
Les services se composent entre eux (par exemple, le service Entreprises interroge le service Offres pour `consulterOffresEntreprise`) et sont orchestrés par le processus BPMN.

Sans état
Chaque requête porte ses informations d'authentification ; aucune session serveur n'est conservée.

Résilience
Tout appel inter-services est protégé par une politique de reprise ; une indisponibilité persistante devient une erreur BPMN traitée par le processus (section 7.4).

Table 2.1 – Propriété des données par service

Service	Type	Tables possédées
1. Étudiants	SOAP	`etudiant`
2. Entreprises	SOAP	`entreprise`
3. Offres de stage	SOAP	`offre`
4. Candidatures	REST	`candidature`, `candidature_historique`
5. Notifications	REST	`notification`
6. Suivi des stages	REST	`stage`, `compte_rendu`, `evaluation`
Orchestrateur	BPMN	tables `ACT_*` du moteur Flowable (base séparée)
Transverse	—	`utilisateur` (comptes et rôles)

2.3 Choix technologiques

2.4 Interactions entre services

Le tableau 2.3 recense les dépendances entre services ; la figure 2.2 montre l'enchaînement des appels lors du dépôt d'une candidature recevable.

2.5 Déploiement

L'ensemble est livré sous la forme d'un **jar exécutable** unique (`java -jar gestion-stages.jar`). Au démarrage, le serveur crée le schéma s'il est absent, charge un jeu de données de démonstration, démarre le moteur BPMN et déploie le processus *avant* d'ouvrir le port, puis publie les points d'accès :

Table 2.2 – Pile technique

Couche	**Technologie**	**Justification**
Langage	Java 21	Recommandé par le sujet ; fils virtuels pour les appels concurrents
SOAP	JAX-WS 4 (Eclipse Metro)	Implémentation de référence, génération automatique du WSDL
REST	JAX-RS 3.1 (Jersey) + Jackson	Implémentation de référence, sérialisation JSON
Serveur	`com.sun.net.httpserver` du JDK	Un seul processus léger, sans conteneur d'applications
Persistance	JDBC, H2 2.3 / PostgreSQL 16	H2 pour la démonstration, PostgreSQL pour la production
BPMN	Flowable 7.1 (moteur embarqué)	Exécute le BPMN 2.0 standard (« outil équivalent » à Bonita)
Interface	HTML, CSS, JavaScript, bpmn-js	Sans framework ni étape de compilation ; diagramme BPMN interactif
E-mails	Jakarta Mail (SMTP) + Mailpit	Vrais e-mails HTML capturés dans une boîte de test locale
Tests	JUnit 5, Postman/Newman, SoapUI	Tests d'intégration de bout en bout
Build	Maven (jar exécutable), Docker Compose	Lancement en une commande

Table 2.3 – Matrice des appels inter-services

Appelant	**Appelé**	**Opérations utilisées**
Orchestrateur BPMN	Étudiants (SOAP)	`verifierEtudiant`, `consulterEtudiant`, `changerStatutEtudiant`
Orchestrateur BPMN	Entreprises (SOAP)	`verifierEntreprise`, `consulterEntreprise`
Orchestrateur BPMN	Offres (SOAP)	`consulterOffre`, `verifierCompatibilite`, `verifierDisponibilite`, `reserverPlace`
Orchestrateur BPMN	Candidatures (REST)	`GET /candidatures?...`, `POST /candidatures`, `PATCH /candidatures/{id}/statut`
Orchestrateur BPMN	Notifications (REST)	`POST /notifications` (7 types de messages)
Orchestrateur BPMN	Stages (REST)	`POST /stages`, `GET /stages?candidatureId=`
Entreprises (SOAP)	Offres (SOAP)	`listerOffres` (offres d'une entreprise, contrôle avant suppression)
Offres (SOAP)	Entreprises (SOAP)	`verifierEntreprise` (référencement avant publication)
Notifications (REST)	Offres, Étudiants (SOAP)	`listerOffresExpirantBientot`, `listerEtudiants`, `verifierCompatibilite` (rappels)
Stages (REST)	Étudiants (SOAP)	`changerStatutEtudiant` (fin de stage)
Candidatures (REST)	Orchestrateur	événement « candidature annulée » (interruption du processus)

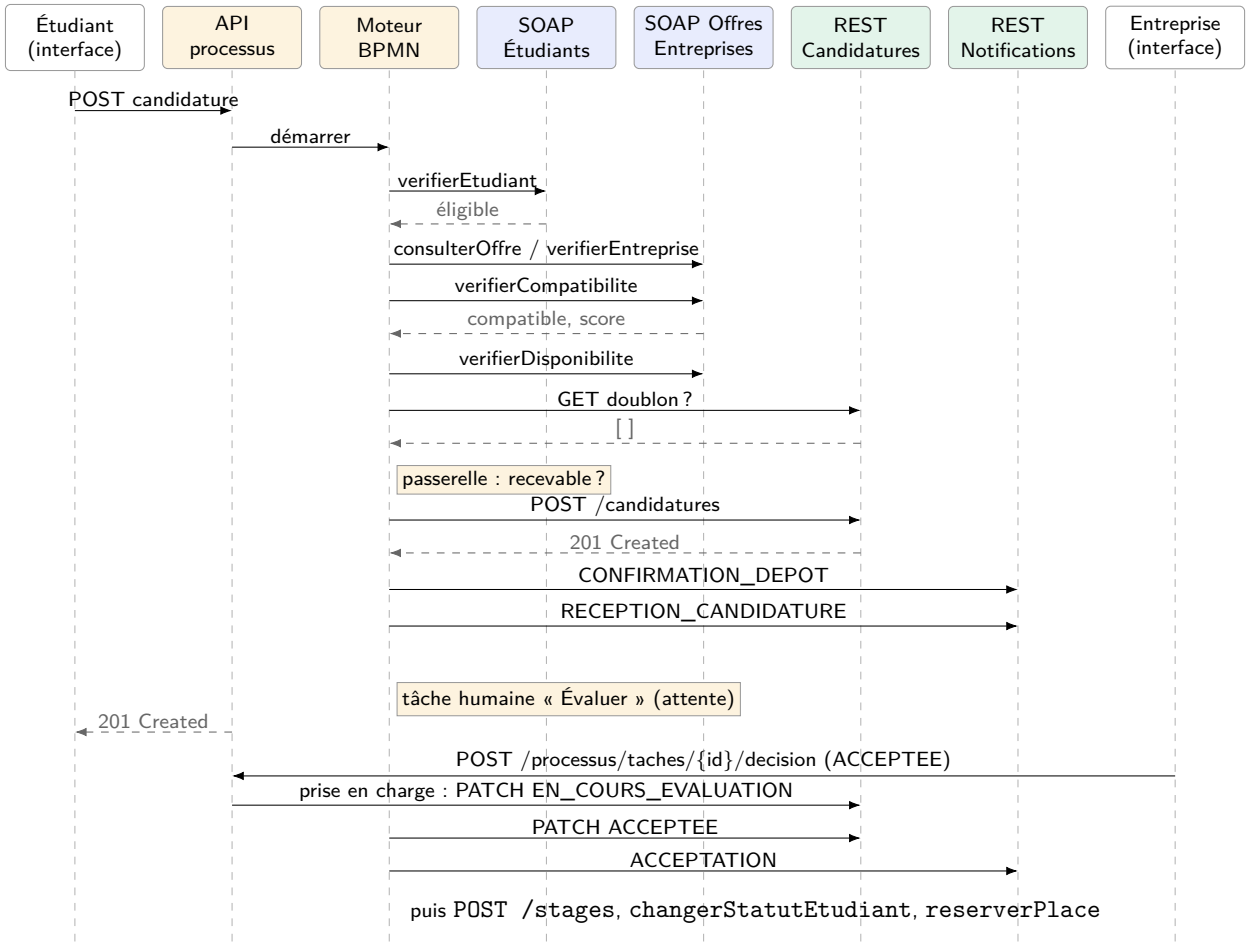


FIGURE 2.2 – Diagramme de séquence du traitement d'une candidature recevable puis acceptée

TABLE 2.4 – Points d'accès publiés

`http://localhost:8090/`	Interface web
`/ws/etudiants?wsdl`, `/ws/entreprises?wsdl`, `/ws/offres?wsdl`	Contrats WSDL des services SOAP
`/api/candidatures`, `/api/notifications`, `/api/stages`	Services REST
`/api/processus`	API d'orchestration (démarrage, tâches humaines, suivi des instances)
`/api/admin/pannes`	Simulateur d'indisponibilité (démonstration)
`http://localhost:8025`	Boîte e-mail de test Mailpit (serveur SMTP sur le port 1025)

La configuration (`application.properties`) est surchargeable par propriété système ou variable d'environnement (`DB_URL`, `SERVER_PORT`…). Un fichier `docker-compose.yml` lance la plateforme avec PostgreSQL et Mailpit. Le port par défaut est `8090` (8080 étant souvent occupé par Jenkins ou Tomcat) ; `-Dserver.port` permet d'en changer.

CHAPITRE 3

Modèle de données

3.1 Schéma relationnel

Le schéma (`schema.sql`) est compatible H2 et PostgreSQL : types standard, clés générées par `GENERATED BY DEFAULT AS IDENTITY`. Les références entre domaines (par exemple `candidature.offre_id`) sont volontairement des **identifiants logiques sans clé étrangère** : chaque service étant propriétaire de ses données, l'intégrité référentielle inter-domaines est garantie par les vérifications du processus, comme elle le serait avec des bases physiquement séparées.

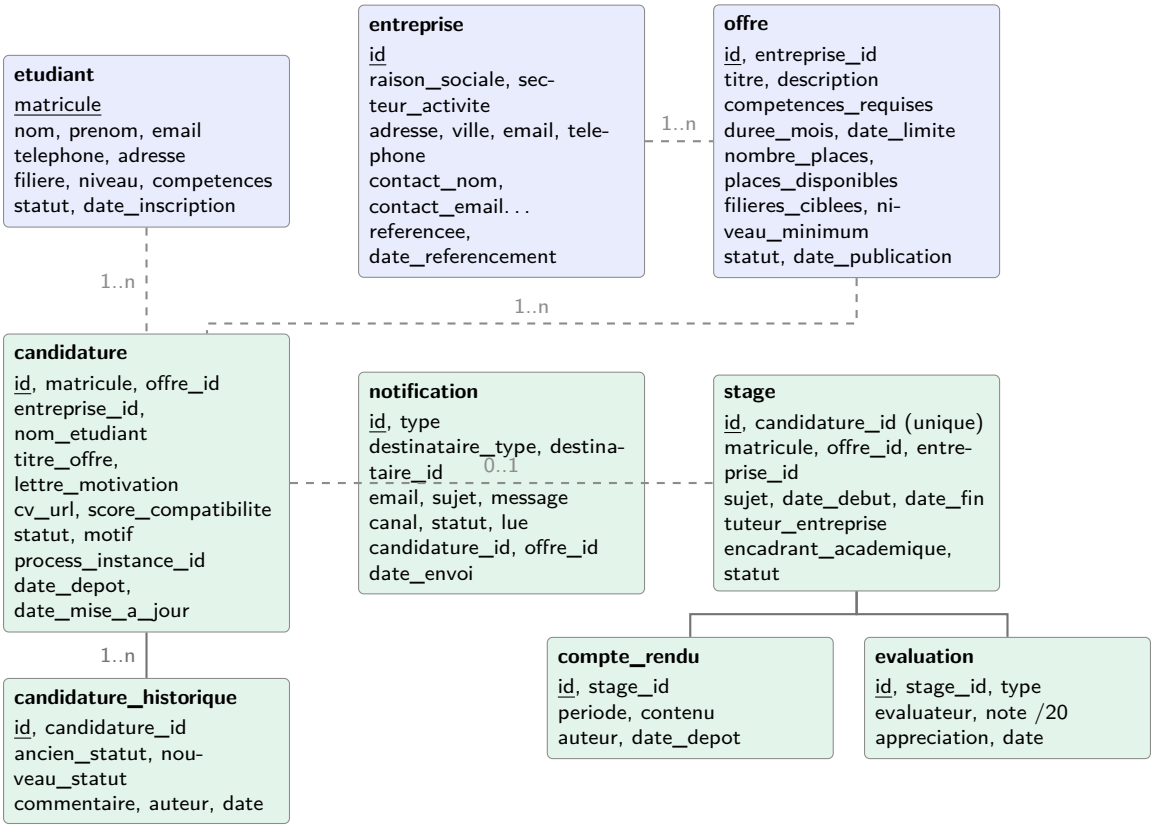


FIGURE 3.1 – Modèle de données (bleu : services SOAP, vert : services REST ; pointillés : références logiques entre domaines)

3.2 Énumérations

3.3 Jeu de données de démonstration

Le jeu de démonstration reprend le contexte de l'INPT : 8 élèves ingénieurs des filières **AMOA, ASEDS, DATA, ICCN, SESNum, SUD** et **Cloud & IoT** (les niveaux suivent la notation LMD : L3 = 1re année, M1 = 2e année, M2 = 3e année), dont une élève suspendue ; 6 entreprises fictives implantées à Casablanca, Rabat, Agadir et Marrakech (dont une non référencée) ; 8 offres, dont une à date limite dépassée et une d'entreprise non référencée. Chaque compte reçoit une

TABLE 3.1 – Valeurs énumérées

Attribut	**Valeurs**
Statut étudiant	`ACTIF`, `SUSPENDU`, `EN_STAGE`, `DIPLOME`
Niveau d'études (ordonné)	`L1` < `L2` < `L3` < `M1` < `M2` < `D`
Statut d'offre	`ACTIVE`, `SUSPENDUE`, `POURVUE`, `CLOTUREE`
Statut de candidature	`DEPOSEE`, `EN_COURS_EVALUATION`, `ACCEPTEE`, `REFUSEE`, `ANNULEE`
Statut de stage	`PLANIFIE`, `EN_COURS`, `TERMINE`, `INTERROMPU`
Type d'évaluation	`MI_PARCOURS`, `FINALE`
Type de notification	`CONFIRMATION_DEPOT`, `RECEPTION_CANDIDATURE`, `REJET_CANDIDATURE`, `ACCEPTATION`, `REFUS`, `DEMANDE_COMPLEMENT`, `RAPPEL_DATE_LIMITE`, `RELANCE_DECISION`, `INCIDENT_TRAITEMENT`, `ANNULATION`, `LIBRE`
Rôle utilisateur	`RESPONSABLE`, `ETUDIANT`, `ENTREPRISE`, `SYSTEME`

clé d'API de démonstration (annexe A.2). Ce jeu couvre volontairement tous les cas de rejet du processus.

CHAPITRE 4

Services SOAP

4.1 Conception commune

Les trois services suivent la même structure, de type *contract-first* par annotations :

— une **interface de service** (SEI) annotée `@WebService`, qui définit le contrat et génère le WSDL ;

— une **implémentation** qui applique le contrôle d'accès, la validation et les règles de gestion ;

— un **DAO** JDBC limité aux tables du service ;

— des **DTO JAXB** (`@XmlType`) portant les types du schéma XSD ; les dates utilisent le type `xs:date`.

Chaque opération déclare une faute métier unique, `ServiceException`, sérialisée en `<detail>` sous la forme d'un élément `ServiceFault` (espace de noms `http://stages.universite.com/commun`) :

TABLE 4.1 – Codes de la faute métier `ServiceFault`

Code	Signification
`DONNEES_INVALIDES`	Échec de validation ; la liste `detail` contient *toutes* les erreurs détectées
`RESSOURCE_INTROUVABLE`	Matricule ou identifiant inconnu
`CONFLIT`	Unicité violée (matricule, e-mail, raison sociale)
`REGLE_METIER`	Règle de gestion non respectée (suppression interdite, plus de place…)
`ACCES_REFUSE`	Rôle ou périmètre de l'appelant insuffisant
`ERREUR_TECHNIQUE`	Service tiers indisponible lors d'une composition

Une requête sans en-tête WS-Security valide est rejetée avant d'atteindre le service par une faute standard `wsse:FailedAuthentication` (chapitre 6). Les WSDL et XSD exportés sont fournis dans `docs/wsdl/`.

4.2 Service 1 — Gestion des étudiants

Adresse : `/ws/etudiants` **Espace de noms** : `http://stages.universite.com/etudiants`

Validation : matricule au format `[A-Z]{1,3}[0-9]{4,10}` (ex. `E2026001`), nom et prénom obligatoires (80 caractères max), e-mail et téléphone au bon format, niveau dans {L1…D}, statut dans l'énumération.

4.3 Service 2 — Gestion des entreprises

Adresse : `/ws/entreprises` **Espace de noms** : `http://stages.universite.com/entreprises`

Table 4.2 – Opérations du service Étudiants

Opération	Entrée	Sortie / règles	Rôles
`ajouterEtudiant`	`Etudiant`	Étudiant créé (statut `ACTIF` par défaut) ; matricule et e-mail uniques	Resp.
`consulterEtudiant`	`matricule`	Fiche complète ; un étudiant ne lit que la sienne ; téléphone et e-mail masqués pour une entreprise	Tous
`modifierEtudiant`	`Etudiant`	Fiche mise à jour, mêmes validations	Resp.
`supprimerEtudiant`	`matricule`	`boolean` ; interdit si `EN_STAGE`	Resp.
`listerEtudiants`	`filiere` ?, `statut` ?	Liste filtrée triée par nom	Resp., Syst.
`verifierEtudiant`	`matricule`	`VerificationEtudiant` : existe, statut, éligible, message	Tous
`changerStatutEtudiant`	`matricule`, `statut`	Fiche mise à jour	Resp., Syst.

Table 4.3 – Opérations du service Entreprises

Opération	Entrée	Sortie / règles	Rôles
`ajouterEntreprise`	`Entreprise`	Entreprise créée ; raison sociale unique ; date de référencement automatique	Resp.
`consulterEntreprise`	`entrepriseId`	Fiche complète (secteur, coordonnées, contact)	Tous
`modifierEntreprise`	`Entreprise`	Une entreprise modifie sa fiche mais pas son référencement	Resp., Ent.
`supprimerEntreprise`	`entrepriseId`	Interdit si l'entreprise a des offres actives	Resp.
`listerEntreprises`	`secteurActivite` ?	Liste filtrée	Tous
`verifierEntreprise`	`entrepriseId`	`VerificationEntreprise` : existe, référencée, message	Tous
`consulterOffresEntreprise`	`entrepriseId`	Liste d'`Offre` — *composition* : appel SOAP au service Offres	Tous

4.4 Service 3 — Gestion des offres de stage

Adresse : `/ws/offres` **Espace de noms** : `http://stages.universite.com/offres`

Compatibilité « intelligente ». `verifierCompatibilite` combine trois critères *bloquants* — filière ciblée (liste vide = toutes), niveau minimum comparé selon l'ordre LMD, et date limite non dépassée — et un **score de compétences** indicatif : pourcentage des compétences requises que possède l'étudiant, comparaison insensible à la casse et aux accents. Le score est conservé avec la candidature et présenté à l'entreprise pour l'aider à trier les dossiers.

1. Une entreprise ne peut créer, modifier ou supprimer que ses propres offres.

Table 4.4 – Opérations du service Offres

Opération	Entrée	Sortie / règles	Rôles
`creerOffre`	`Offre`	Statut `ACTIVE` ; date limite future ; entreprise référencée (appel SOAP Entreprises)	Resp., Ent.[1]
`consulterOffre`	`offreId`	Offre complète	Tous
`modifierOffre`	`Offre`	Places ≥ places déjà attribuées ; propriétaire et date de publication figés	Resp., Ent.
`supprimerOffre`	`offreId`	Interdit si des stagiaires ont été retenus	Resp., Ent.
`listerOffres`	`statut` ?, `entrepriseId` ?	Liste triée par date limite	Tous
`listerOffresDisponibles`	—	Offres actives, non expirées, avec places	Tous
`verifierDisponibilite`	`offreId`	`DisponibiliteOffre` : active, date dépassée, places, message	Tous
`verifierCompatibilite`	`offreId`, `filiere`, `niveau`, `competences`	`Compatibilite` : filière, niveau, date limite, score de compétences, manques, motifs	Tous
`reserverPlace`	`offreId`	Décrément atomique ; passage à `POURVUE` à zéro	Resp., Syst.
`listerOffresExpirantBientot`	`jours`	Offres actives clôturant dans l'intervalle	Tous

Réservation sans sur-attribution. `reserverPlace` exécute `UPDATE offre SET places_disponibles = places_disponibles - 1 ... WHERE id = ? AND places_disponibles > 0` : la condition dans la clause `WHERE` rend l'opération atomique, même si deux acceptations sont simultanées.

4.5 Extrait de WSDL

Listing 4.1 – Extrait de `etudiants.wsdl` : opération `verifierEtudiant`

```
<message name="verifierEtudiant">
  <part name="parameters" element="tns:verifierEtudiant"/>
</message>
<message name="ServiceException">
  <part name="fault" element="ns1:ServiceFault"/>
</message>
<portType name="EtudiantService">
  <operation name="verifierEtudiant">
    <input  message="tns:verifierEtudiant"/>
    <output message="tns:verifierEtudiantResponse"/>
    <fault  message="tns:ServiceException" name="ServiceException"/>
  </operation>
</portType>
<binding name="EtudiantPortBinding" type="tns:EtudiantService">
  <soap:binding transport="http://schemas.xmlsoap.org/soap/http" style="document"/>
  <operation name="verifierEtudiant">
    <soap:operation soapAction=""/>
    <input><soap:body use="literal"/></input>
```

```
    <output><soap:body use="literal"/></output>
    <fault name="ServiceException"><soap:fault name="ServiceException" use="literal"/></fault>
  </operation>
</binding>
<service name="EtudiantService">
  <port name="EtudiantPort" binding="tns:EtudiantPortBinding">
    <soap:address location="http://localhost:8090/ws/etudiants"/>
  </port>
</service>
```

Listing 4.2 – Type XSD `VerificationEtudiant` généré

```
<xs:complexType name="VerificationEtudiant">
  <xs:sequence>
    <xs:element name="matricule" type="xs:string" minOccurs="0"/>
    <xs:element name="existe"    type="xs:boolean"/>
    <xs:element name="statut"    type="xs:string" minOccurs="0"/>
    <xs:element name="eligible"  type="xs:boolean"/>
    <xs:element name="message"   type="xs:string" minOccurs="0"/>
  </xs:sequence>
</xs:complexType>
```

4.6 Exemples d'échanges

Les échanges suivants ont été obtenus sur la plateforme en fonctionnement (fichiers `docs/soap/`).

Listing 4.3 – Requête `verifierCompatibilite` avec en-tête WS-Security

```
<soapenv:Envelope xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope/"
    xmlns:off="http://stages.universite.com/offres"
    xmlns:wsse="http://docs.oasis-open.org/wss/2004/01/oasis-200401-wss-wssecurity-secext-1.0.xsd
    ">
  <soapenv:Header>
    <wsse:Security soapenv:mustUnderstand="1">
      <wsse:UsernameToken>
        <wsse:Username>admin</wsse:Username>
        <wsse:Password>Admin@2026</wsse:Password>
      </wsse:UsernameToken>
    </wsse:Security>
  </soapenv:Header>
  <soapenv:Body>
    <off:verifierCompatibilite>
      <offreId>1</offreId> <filiere>AMOA</filiere> <niveau>M1</niveau>
      <competences>BPMN, UML, Java</competences>
    </off:verifierCompatibilite>
  </soapenv:Body>
</soapenv:Envelope>
```

Listing 4.4 – Réponse obtenue

```
<S:Envelope xmlns:S="http://schemas.xmlsoap.org/soap/envelope/">
  <S:Body>
    <ns2:verifierCompatibiliteResponse xmlns:ns2="http://stages.universite.com/offres">
      <compatibilite>
        <offreId>1</offreId>
        <compatible>true</compatible>
        <filiereCompatible>true</filiereCompatible>
        <niveauCompatible>true</niveauCompatible>
        <scoreCompetences>50</scoreCompetences>
        <competenceManquante>SQL</competenceManquante>
        <competenceManquante>Gestion de projet</competenceManquante>
```

```
      </compatibilite>
    </ns2:verifierCompatibiliteResponse>
  </S:Body>
</S:Envelope>
```

Listing 4.5 – SOAP Fault de validation (`ajouterEtudiant` avec données invalides)

```
<S:Fault>
  <faultcode>S:Server</faultcode>
  <faultstring>Données invalides</faultstring>
  <detail>
    <ns3:ServiceFault xmlns:ns3="http://stages.universite.com/commun">
      <code>DONNEES_INVALIDES</code>
      <message>Données invalides</message>
      <detail>Le champ 'matricule' est invalide (attendu : lettres majuscules suivies de chiffres
    , ex. E2026001)</detail>
      <detail>Le champ 'email' est invalide (attendu : adresse e-mail)</detail>
      <detail>Le champ 'niveau' doit valoir l'une des valeurs [L1, L2, L3, M1, M2, D]</detail>
    </ns3:ServiceFault>
  </detail>
</S:Fault>
```

4.7 Implémentation

Listing 4.6 – Contrat (extrait) et implémentation de `verifierEtudiant`

```
@WebService(name = "EtudiantService", targetNamespace = Namespaces.ETUDIANTS)
public interface EtudiantService {
    @WebMethod(operationName = "verifierEtudiant")
    @WebResult(name = "verification")
    VerificationEtudiant verifierEtudiant(@WebParam(name = "matricule") String matricule)
            throws ServiceException;
    // ...
}

@Override
public VerificationEtudiant verifierEtudiant(String matricule) throws ServiceException {
    SoapSecurity.utilisateur(context);                // appelant authentifié (WS-Security)
    if (matricule == null || matricule.isBlank()) {
        throw ServiceException.invalide(List.of("Le matricule est obligatoire"));
    }
    return dao.trouver(matricule)
            .map(e -> {
                boolean eligible = Etudiant.ACTIF.equals(e.getStatut());
                String msg = eligible ? "Étudiant actif, éligible aux candidatures"
                        : "Étudiant non éligible (statut " + e.getStatut() + ")";
                return new VerificationEtudiant(matricule, true, e.getStatut(), eligible, msg);
            })
            .orElseGet(() -> new VerificationEtudiant(matricule, false, null, false,
                    "Aucun étudiant ne correspond au matricule " + matricule));
}
```

CHAPITRE 5

Services REST

5.1 Conventions communes

— **Ressources** nommées au pluriel (`/candidatures`, `/stages`), sous-ressources pour les compositions (`/stages/{id}/evaluations`), filtres en paramètres de requête (`?matricule=&statut=`).

— **Méthodes HTTP** : `GET` lecture (sûre, idempotente), `POST` création, `PATCH` modification partielle (statut, complément), `DELETE` annulation.

— **Format** : JSON UTF-8, dates ISO-8601, champs nuls omis.

— **Création** : `201 Created` avec l'en-tête `Location` de la ressource créée.

— **Authentification** par clé d'API (en-tête `X-API-Key`) pour les applications et scripts, ou HTTP Basic pour l'interface web ; le périmètre est restreint automatiquement selon l'acteur (un étudiant ne voit que ses candidatures, une entreprise que celles qui la concernent).

Table 5.1 – Codes HTTP retournés

Code	Erreur	Cas
200 OK	—	Lecture, modification, annulation réussies
201 Created	—	Création (candidature, notification, stage, compte rendu, évaluation, instance de processus)
400 Bad Request	`DONNEES_INVALIDES`, `JSON_INVALIDE`	Champs obligatoires manquants, format ou bornes non respectés, JSON mal formé
401 Unauthorized	`NON_AUTHENTIFIE`, `CLE_API_INVALIDE`	Identifiants ou clé d'API absents, invalides ou révoqués
403 Forbidden	`ACCES_REFUSE`	Rôle non autorisé ou ressource hors périmètre
404 Not Found	`RESSOURCE_INTROUVABLE`	Identifiant inconnu
409 Conflict	`CONFLIT`, `TRANSITION_INTERDITE`, `DECISION_VIA_PROCESSUS`	Doublon, transition d'état interdite, décision contournant le processus
422 Unprocessable Entity	`REGLE_METIER`	Règle de gestion (annulation d'une candidature déjà traitée…)
500 / 502	`ERREUR_INTERNE`, `ERREUR_SERVICE_AMONT`	Erreur inattendue, faute d'un service SOAP appelé
503 Service Unavailable	`SERVICE_INDISPONIBLE`	Service indisponible (en-tête `Retry-After`)

Toutes les erreurs ont le même corps JSON, produit par un `ExceptionMapper` unique :

Listing 5.1 – Corps d'erreur uniforme (400)

```
{
```

```
  "status": 400,
  "erreur": "DONNEES_INVALIDES",
  "message": "Données invalides",
  "details": [ "Le champ 'matricule' est obligatoire", "Le champ 'offreId' est obligatoire" ],
  "horodatage": "2026-10-23T18:22:16.985909069Z"
}
```


5.2 Service 4 — Gestion des candidatures

Base : `/api/candidatures`

TABLE 5.2 – Points d'accès du service Candidatures

Méthode	URI	Description	Codes
POST	`/candidatures`	Dépôt (enregistrement) d'une candidature ; refus des doublons et au-delà de 5 candidatures en cours. Réservé à l'orchestrateur	201, 400, 409, 422
GET	`/candidatures`	Recherche (`matricule`, `offreId`, `entrepriseId`, `statut`)	200
GET	`/candidatures/etudiant/{matricule}`	Candidatures d'un étudiant	200, 403
GET	`/candidatures/{id}`	Détail	200, 403, 404
GET	`/candidatures/{id}/historique`	Historique horodaté des changements de statut	200, 404
PATCH	`/candidatures/{id}`	Complément du dossier (lettre, CV) par l'étudiant	200, 422
PATCH	`/candidatures/{id}/statut`	Changement de statut `{statut, motif}` contrôlé par l'automate	200, 400, 409
DELETE	`/candidatures/{id}`	Annulation (statut `ANNULEE`), uniquement tant que la candidature est `DEPOSEE`	200, 403, 422

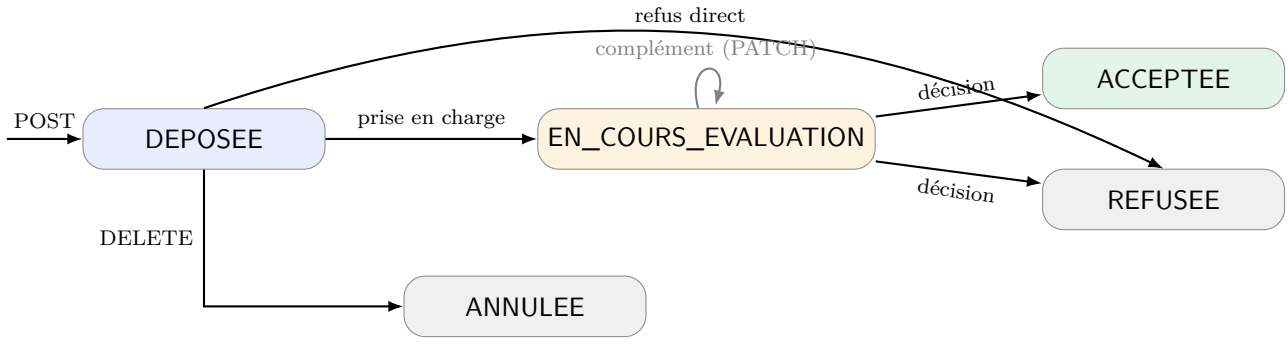


FIGURE 5.1 – Automate des statuts d'une candidature (états finals en gris et vert)

Listing 5.2 – Ressource candidature (GET `/api/candidatures/1`)


```
{
  "id": 1,
  "matricule": "E2026001",
```

```
  "nomEtudiant": "Yasmine Benali",
  "offreId": 1,
  "titreOffre": "Assistant chef de projet AMOA",
  "entrepriseId": 1,
  "lettreMotivation": "Très motivée par la modélisation des processus métier.",
  "cvUrl": "https://cv.example/E2026001.pdf",
  "scoreCompatibilite": 100,
  "statut": "ACCEPTEE",
  "motif": "Bienvenue !",
  "processInstanceId": "5",
  "dateDepot": "2026-10-23T18:18:00.699389",
  "dateMiseAJour": "2026-10-23T18:18:03.470788"
}
```


5.3 Service 5 — Notifications

Base : `/api/notifications`

Table 5.3 – Points d'accès du service Notifications

Méthode	URI	Description	Codes
POST	`/notifications`	Envoi d'une notification typée `{type, destinataireType, destinataireId, email, candidatureId, parametres}` ; le gabarit est rendu côté serveur	201, 400
GET	`/notifications`	Boîte de réception (filtres `destinataireType`, `destinataireId`, `candidatureId`, `type`)	200
GET	`/notifications/{id}`	Détail (destinataire uniquement)	200, 403, 404
PATCH	`/notifications/{id}/lue`	Marquer comme lue	200
POST	`/notifications/rappels`	Campagne de rappels des dates limites `{joursAvant}`	200, 400

Les messages sont produits à partir de gabarits paramétrés (`{nomEtudiant}`, `{titreOffre}`, `{motif}`…). Les types couvrent les besoins du sujet : confirmation de dépôt, information de l'entreprise, acceptation, refus, demande de complément, rappels de date limite, ainsi que le rejet automatique, la relance de l'entreprise et l'information en cas d'incident.

Le canal e-mail envoie de **vrais messages** en SMTP (Jakarta Mail), au format texte et HTML aux couleurs de l'INPT. En démonstration, le serveur SMTP est **Mailpit** : il capture tous les messages dans une boîte de réception consultable sur `http://localhost:8025`, sans qu'aucun e-mail ne sorte de la machine (figure 10.4). Chaque notification reste archivée en base avec son statut d'envoi (`ENVOYEE`, `ECHEC_ENVOI` si le serveur SMTP ne répond pas, `JOURNALISEE` si l'envoi est désactivé), ce qui la rend aussi consultable dans l'interface. Pour un vrai serveur, il suffit de changer `mail.smtp.hote`.

La **campagne de rappels** illustre la composition de services : le service REST interroge le service SOAP Offres (`listerOffresExpirantBientot`), le service SOAP Étudiants (`listerEtudiants(statut=ACTIF)`) puis `verifierCompatibilite` pour chaque couple, et n'envoie qu'un rappel par étudiant et par offre. Elle s'exécute automatiquement toutes les 24 heures et peut être déclenchée à la demande.

5.4 Service 6 — Suivi des stages

Base : `/api/stages`

Table 5.4 – Points d'accès du service Suivi des stages

Méthode	URI	Description	Codes
POST	`/stages`	Enregistrement d'un stage accepté (un seul par candidature)	201, 400, 409
GET	`/stages`	Recherche (`matricule`, `entrepriseId`, `statut`, `candidatureId`)	200
GET	`/stages/{id}`	Détail	200, 403, 404
GET	`/stages/{id}/statut`	Statut, dates, durée, jours restants et progression calculée	200
PATCH	`/stages/{id}/statut`	Transition `PLANIFIE→EN_COURS→TERMINE`, ou `INTERROMPU`	200, 409
POST	`/stages/{id}/comptes-rendus`	Dépôt d'un compte rendu périodique	201, 400, 422
GET	`/stages/{id}/comptes-rendus`	Liste des comptes rendus	200
POST	`/stages/{id}/evaluations`	Évaluation (`MI_PARCOURS` ou `FINALE`, note sur 20)	201, 400, 409, 422
GET	`/stages/{id}/evaluations`	Liste des évaluations	200
GET	`/stages/etudiant/{matricule}/historique`	Stages réalisés, nombre de comptes rendus, évaluations et moyenne	200, 403

Listing 5.3 – GET `/api/stages/1/statut`

```
{ "stageId": 1, "statut": "TERMINE", "dateDebut": "2027-02-01", "dateFin": "2027-07-31",
  "dureeJours": 180, "joursRestants": 281, "progression": 100 }
```

L'évaluation finale clôture le stage (`TERMINE`) ; à la fin ou à l'interruption d'un stage, le service appelle `changerStatutEtudiant(ACTIF)` du service SOAP Étudiants pour rendre l'étudiant à nouveau éligible.

5.5 API d'orchestration

Base : `/api/processus`. Elle expose le processus BPMN aux acteurs humains.

Table 5.5 – Points d'accès de l'orchestration

Méthode	URI	Description	Rôles
POST	`/processus/candidatures`	Soumission : démarre une instance ; renvoie son état (rejet motivé, attente de décision ou incident)	Étud., Resp.
GET	`/processus/taches`	Tâches humaines en attente (filtrées par entreprise)	Ent., Resp.
POST	`/processus/taches/{id}/prise-en-charge`	L'entreprise commence l'examen : tâche attribuée, candidature `EN_COURS_EVALUATION`	Ent., Resp.
POST	`/processus/taches/{id}/decision`	Décision `ACCEPTEE` / `REFUSEE` / `COMPLEMENT` + commentaire, dates, tuteur	Ent., Resp.
POST	`/processus/taches/{id}/incident`	Traitement d'un incident : `RELANCER` ou `ABANDONNER`	Resp.
GET	`/processus/instances`	Instances récentes	Resp.
GET	`/processus/instances/{id}`	État, variables, tâches, activités actives et trace d'exécution	Concernés
GET	`/processus/definition`	Modèle BPMN 2.0 (XML)	Tous

CHAPITRE 6

Sécurité des échanges

6.1 Authentification

Services SOAP — WS-Security
Chaque requête doit porter un en-tête OASIS WS-Security 1.0 `<wsse:Security><wsse:UsernameToken>` déclaré `mustUnderstand`. Un intercepteur JAX-WS (`SoapSecurityHandler`) extrait le jeton, authentifie l'appelant et dépose l'utilisateur dans le contexte du message. En cas d'échec, la requête n'atteint pas le service : la réponse est une SOAP Fault `wsse:FailedAuthentication`. Le WSDL reste consultable sans authentification.

Services REST — clé d'API ou HTTP Basic
Un filtre JAX-RS (`AuthentificationFilter`, priorité `AUTHENTICATION`) accepte deux schémas : une **clé d'API** dans l'en-tête `X-API-Key`, destinée aux applications (orchestrateur, Postman, scripts), ou HTTP Basic, utilisé par l'interface web après connexion. Une clé invalide ou révoquée est refusée (`401 CLE_API_INVALIDE`). Seul `/api/sante` est public.

Gestion des clés
Seule l'empreinte SHA-256 de la clé est stockée. `POST /api/moi/cle-api` (bouton « Ma clé d'API » de l'interface) génère une nouvelle clé aléatoire de 256 bits, l'affiche une seule fois et révoque la précédente.

Appels inter-services
L'orchestrateur et les services composites utilisent un compte technique de rôle `SYSTEME`, configurable, injecté par un intercepteur client (SOAP) ou par sa clé d'API (REST).

6.2 Protection des informations sensibles

— **Mots de passe** hachés par PBKDF2-HMAC-SHA256 (120 000 itérations, sel aléatoire de 128 bits) ; la comparaison s'effectue en temps constant (`MessageDigest.isEqual`). Seule une empreinte SHA-256 du couple identifiant/mot de passe est gardée en cache mémoire pour éviter de refaire la dérivation à chaque appel.

— **Masquage des données personnelles** : lorsqu'une entreprise consulte la fiche d'un candidat (`consulterEtudiant`), elle voit son profil académique (filière, niveau, compétences) mais le téléphone et l'e-mail sont masqués (`+212 6 •• •• •• 01`, `y•••@...`) et l'adresse n'est pas transmise. Les adresses e-mail sont également masquées dans les journaux du serveur.

— **Cloisonnement des données** : un étudiant ne peut lire que sa fiche, ses candidatures, ses notifications et ses stages ; une entreprise que ses offres, les candidatures et stages qui la concernent, et elle ne peut pas modifier son propre référencement. Les filtres de recherche sont *imposés* côté serveur, indépendamment des paramètres reçus.

— **Variables de processus** : l'API d'orchestration n'expose qu'une liste blanche de variables.

— **Requêtes SQL paramétrées** partout (`PreparedStatement`), ce qui exclut l'injection SQL.

— **Interface web** : toutes les données affichées sont échappées (prévention XSS), l'en-tête `X-Content-Type-Options: nosniff` est envoyé et les identifiants ne sont conservés qu'en `sessionStorage`.

6.3 Contrôle d'accès par rôle

TABLE 6.1 – Matrice des droits (L : lecture, E : écriture, P : périmètre propre uniquement)

Ressource / opération	Étudiant	Entreprise	Responsable	Système
Fiche étudiant (consulter)	L (P)	L	L/E	L/E statut
Entreprise (fiche)	L	L/E (P, hors référencement)	L/E	L
Offres (créer, modifier, supprimer)	L	L/E (P)	L/E	L, réserver
Soumettre une candidature (processus)	E (P)	—	E	—
Candidatures (consulter)	L (P)	L (P)	L	L/E
Annuler / compléter une candidature	E (P)	—	E	E
Décision sur une candidature	—	E (P)	E	—
Notifications	L (P)	L (P)	L/E	E
Stages, comptes rendus	L/E (P)	L/E (P)	L/E	E
Évaluations	L (P)	E (P)	E	—
Incidents, instances, pannes simulées	—	—	L/E	—

6.4 Confidentialité du transport

UsernameToken en `PasswordText` et HTTP Basic transmettent le mot de passe encodé mais non chiffré : en production, la plateforme doit être exposée exclusivement en **HTTPS** (terminaison TLS par un proxy inverse ou `HttpsServer` du JDK), ce qui ne demande aucune modification des services. Le contexte de sécurité REST signale d'ailleurs si la requête est sécurisée (`isSecure()`). Des évolutions possibles sont présentées au chapitre 11 (jetons JWT/OAuth 2, signature WS-Security).

CHAPITRE 7

Orchestration BPMN

7.1 Outil et principe

Le processus « Traitement d'une candidature de stage » est modélisé en **BPMN 2.0** standard (`candidature-stage.bpmn20.xml`, modèle et diagramme BPMN-DI) et exécuté par le moteur **Flowable**, embarqué dans la plateforme. Flowable est l'« outil équivalent » à Bonita autorisé par le sujet : il exécute le même standard, ce qui permet de démontrer une orchestration *réellement exécutée* et testée automatiquement. Le fichier reste importable dans Bonita Studio, Camunda Modeler ou bpmn.io (procédure en section 7.6).

Chaque **tâche de service** est reliée à un délégué Java qui appelle un service SOAP ou REST *par le réseau*, exactement comme le ferait un connecteur Bonita. Les tâches humaines (décision de l'entreprise, traitement d'un incident) sont exposées par l'API `/api/processus` et l'interface web.

7.2 Structure du processus

Le pool « Plateforme de gestion des stages » comporte quatre couloirs : *Étudiant*, *Orchestrateur SOA*, *Entreprise partenaire* et *Responsable des stages*. Les étapes de l'orchestrateur sont regroupées en trois **sous-processus** qui délimitent les zones de reprise en cas d'erreur.

7.3 Passerelles et conditions

La variable `recevable` n'est vraie que si l'étudiant est actif, l'offre active, l'entreprise référencée, la filière et le niveau compatibles, la date limite non dépassée, une place disponible, aucune candidature non annulée n'existe pour le même couple étudiant/offre, et l'étudiant a moins de 5 candidatures en cours. Tous les motifs de refus sont accumulés dans `motifRejet` et repris dans la notification adressée à l'étudiant.

7.4 Gestion des erreurs et des indisponibilités

La gestion des erreurs est organisée en quatre niveaux.

1. Reprise automatique. Tout appel inter-services passe par `Resilience.avecReprise` : 3 tentatives avec un délai exponentiel (400 ms, 800 ms), délais de connexion et de réponse bornés à 5 s. Seules les erreurs *techniques* sont rejouées (connexion impossible, délai dépassé, HTTP 5xx) ; une erreur *fonctionnelle* (HTTP 4xx, SOAP Fault métier) est propagée immédiatement.

2. Erreur BPMN et événement de bordure. Si le service reste indisponible, le délégué lève `BpmnError("ERREUR_TECHNIQUE")` après avoir renseigné `serviceEnErreur`, `messageErreur` et `activiteEnErreur`. Chacun des trois sous-processus porte un **événement d'erreur de bordure** « Service indisponible » qui interrompt le sous-processus et dirige le flux vers « Journaliser l'incident ».

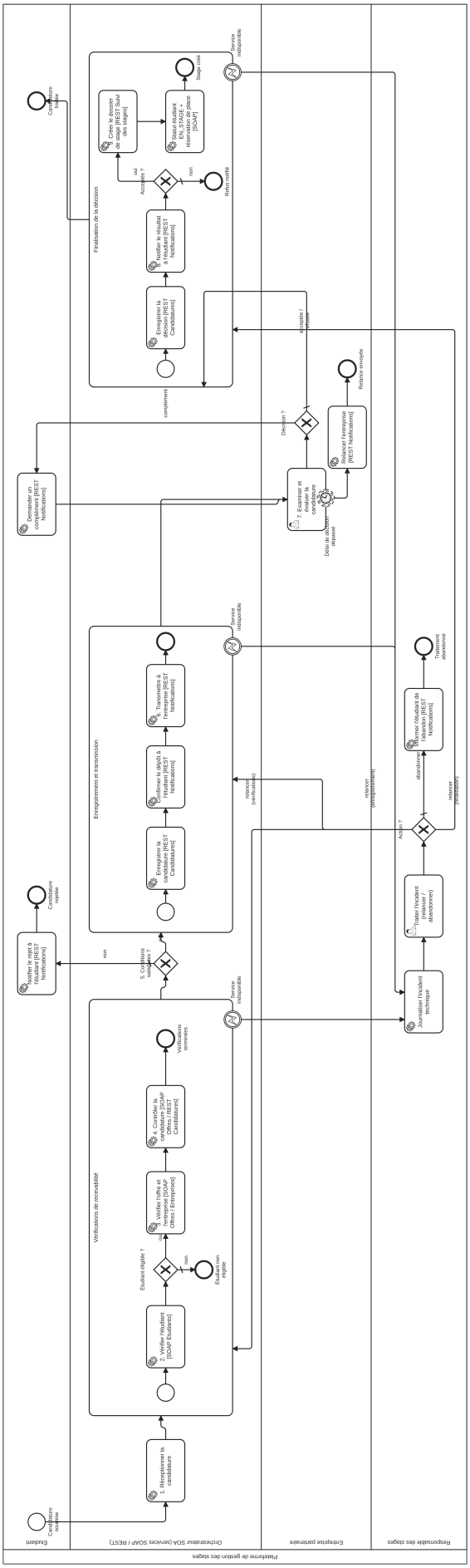


FIGURE 7.1 – Diagramme BPMN du processus de traitement d'une candidature (rendu bpmn-js du fichier exécuté)

Table 7.1 – Correspondance entre les étapes du sujet et le modèle BPMN

Étape du sujet	**Éléments BPMN**	**Service appelé**
1. Réception	Événement de début « Candidature soumise » ; tâche « Réceptionner » (contrôle de forme, initialisation)	API `POST /processus/candidatures`
2. Vérification de l'étudiant	Tâche « Vérifier l'étudiant » ; passerelle « Étudiant éligible ? »	SOAP Étudiants : `verifierEtudiant`, `consulterEtudiant`
3. Vérification de l'offre	Tâche « Vérifier l'offre et l'entreprise »	SOAP Offres : `consulterOffre`, `verifierCompatibilite` ; SOAP Entreprises : `verifierEntreprise`, `consulterEntreprise`
4. Contrôle de la candidature	Tâche « Contrôler la candidature »	SOAP Offres : `verifierDisponibilite` ; REST : `GET /candidatures` (doublon)
5. Passerelle de décision	Passerelle exclusive « Conditions satisfaites ? » : rejet notifié *ou* enregistrement	REST : `POST /notifications` *ou* `POST /candidatures`
6. Transmission à l'entreprise	Sous-processus « Enregistrement et transmission » : confirmation à l'étudiant, notification de l'entreprise	REST Notifications
7. Décision de l'entreprise	Tâche humaine « Examiner et évaluer la candidature » : prise en charge (statut « en cours d'évaluation »), puis passerelle « Décision ? » (accepter, refuser, demander un complément)	API `.../prise-en-charge` puis `.../decision` ; REST `PATCH /candidatures/{id}/statut`
8. Notification du résultat	Tâche « Notifier le résultat » (gabarit ACCEPTATION ou REFUS)	REST Notifications
9. Mise à jour du suivi	Passerelle « Acceptée ? » ; « Créer le dossier de stage » ; « Statut EN_STAGE + réservation de place »	REST : `POST /stages` ; SOAP : `changerStatutEtudiant`, `reserverPlace`

Table 7.2 – Passerelles exclusives et expressions de condition

Passerelle	**Branches (condition JUEL)**	**Défaut**
Étudiant éligible ?	oui : `${etudiantEligible}`	fin « Étudiant non éligible »
Conditions satisfaites ?	oui : `${recevable}` → enregistrement	notification du rejet
Décision ?	complément : `${decision == 'COMPLEMENT'}` → notification puis retour à l'évaluation	finalisation
Acceptée ?	oui : `${decision == 'ACCEPTEE'}` → création du stage	fin « Refus notifié »
Action ? (incident)	relancer : `${actionIncident == 'RELANCER' && etapeCourante == '...'}` (une branche par sous-processus)	abandon notifié

3. Traitement humain de l'incident. La tâche « Traiter l'incident » est attribuée au responsable des stages, qui choisit de **relancer** l'étape — le flux revient au début du sous-processus en erreur, identifié par la variable `etapeCourante` positionnée par un écouteur d'exécution — ou d'**abandonner**, auquel cas l'étudiant est informé et l'instance se termine par l'événement « Traitement abandonné ».

4. Notifications non bloquantes. Une notification non envoyée ne justifie pas de bloquer un étudiant : après les reprises, l'échec est consigné dans la variable `incidents` et le processus continue.

Listing 7.1 – Conversion d'une indisponibilité en erreur BPMN (`AbstractServiceDelegate`)

```
@Override
public final void execute(DelegateExecution execution) {
    try {
        executer(execution);
    } catch (ServiceIndisponibleException e) {
        String msg = "Service « " + e.getService() + " » indisponible : " + e.getMessage();
        execution.setVariable("serviceEnErreur", e.getService());
        execution.setVariable("messageErreur", msg);
        execution.setVariable("activiteEnErreur", execution.getCurrentActivityId());
        throw new BpmnError(ERREUR_TECHNIQUE, msg);   // intercepté par l'événement de bordure
    }
}
```

Idempotence des reprises. Relancer un sous-processus ne doit pas dupliquer les effets déjà produits : l'enregistrement de la candidature est ignoré si `candidatureId` est déjà connu, la création du stage récupère le stage existant en cas de réponse `409`, et la réservation de place est mémorisée par `placeReservee`.

Démonstration. Le simulateur de pannes (`POST /api/admin/pannes` ou écran « Supervision ») rend n'importe lequel des six services indisponible (HTTP 503) ; les scénarios PT-04 à PT-06 du chapitre 9 vérifient automatiquement ces comportements.

7.5 Autres mécanismes

Minuteur de relance
Un événement minuteur de bordure **non interrompant** est attaché à « Évaluer la candidature ». Passé le délai (`P3D` par défaut, paramétrable), l'entreprise est relancée par une notification `RELANCE_DECISION` sans que la tâche d'évaluation soit annulée.

Prise en charge
La candidature reste `DEPOSEE` — donc annulable par l'étudiant — tant que l'entreprise n'a pas ouvert le dossier. Le bouton « Examiner » attribue la tâche à l'entreprise (*claim*) et fait passer la candidature `EN_COURS_EVALUATION` ; une décision rendue directement déclenche cette prise en charge.

Demande de complément
La branche « complément » de la passerelle « Décision ? » notifie l'étudiant puis revient à la tâche d'évaluation ; l'étudiant complète son dossier par `PATCH /candidatures/{id}`.

Annulation
Lorsqu'un étudiant annule sa candidature, le service Candidatures publie un événement « candidature annulée » ; l'orchestrateur, abonné, interrompt l'instance correspondante (statut `INTERROMPU`).

Cohérence
Une décision ne peut pas contourner le processus : un `PATCH` manuel vers `ACCEPTEE` ou `REFUSEE` sur une candidature orchestrée est refusé (`409 DECISION_VIA_PROCESSUS`).

Traçabilité
L'historique du moteur est exposé par `GET /processus/instances/{id}` : activités traversées avec horodatage et durée, activités actives, variables, raison d'arrêt. L'interface superpose ces informations au diagramme (figure 10.6).

7.6 Reproduction dans Bonita Studio

Le guide `docs/bonita/GUIDE_BONITA.md` détaille la reproduction du processus dans Bonita :

1. importer `docs/bonita/traitement-candidature-bonita.bpmn` (*Fichier › Importer › BPMN 2.0*), variante préparée pour Bonita : flux à plat (Bonita ne gère pas les sous-processus embarqués), un événement d'erreur de bordure sur chaque tâche critique, aucune extension propriétaire, noms de tâches courts adaptés à la taille fixe des tâches de Bonita. L'import a été vérifié dans Bonita Studio ;
2. déclarer les variables du processus (tableau du guide) ;
3. associer à chaque tâche de service un connecteur *Web Service SOAP* ou *REST* (même opérations qu'au tableau 7.1, compte `systeme`) ;
4. configurer *Lever un événement d'erreur* en cas d'échec des connecteurs critiques, et *Ignorer l'erreur* pour les notifications ;
5. définir les acteurs *entreprise* et *responsable* ainsi que le minuteur non interrompant.

CHAPITRE 8

Règles de gestion

Les règles ci-dessous sont implémentées dans les services indiqués ; la colonne « Test » renvoie aux scénarios automatisés du chapitre 9 qui les vérifient.

TABLE 8.1 – Règles de gestion

N°	Règle	Service	Test
Étudiants			
RG-01	Le matricule est unique et respecte le format `[A-Z]{1,3}[0-9]{4,10}`.	Étudiants	ST-04, ST-05
RG-02	L'adresse e-mail d'un étudiant est unique (insensible à la casse).	Étudiants	—
RG-03	Un étudiant créé sans statut est `ACTIF`.	Étudiants	ST-03
RG-04	Seul un étudiant `ACTIF` est éligible aux candidatures (un étudiant suspendu, en stage ou diplômé ne l'est pas).	Étudiants, BPMN	ST-07, PT-02
RG-05	Un étudiant en stage ne peut pas être supprimé.	Étudiants	—
RG-06	Un étudiant ne consulte que sa propre fiche et ne modifie aucune donnée de référence.	Étudiants	ST-06
RG-32	Une entreprise ne voit pas les coordonnées personnelles d'un candidat (téléphone et e-mail masqués, adresse non transmise).	Étudiants	ST-12
Entreprises et offres			
RG-07	La raison sociale d'une entreprise est unique ; la date de référencement est fixée à l'activation du référencement.	Entreprises	ST-08
RG-08	Une entreprise ayant des offres actives ne peut pas être supprimée.	Entreprises	ST-08
RG-09	Seule une entreprise référencée peut publier une offre.	Offres	ST-09
RG-10	Une entreprise ne gère que ses propres offres et ne modifie pas son référencement.	Offres, Entreprises	ST-09
RG-11	À la création, la date limite est future, la durée comprise entre 1 et 12 mois, le nombre de places positif ; l'offre est `ACTIVE` et toutes ses places sont disponibles.	Offres	ST-09
RG-12	Le nombre de places d'une offre ne peut descendre sous le nombre de places déjà attribuées.	Offres	—
RG-13	Une offre dont des stagiaires ont été retenus ne peut être supprimée (seulement clôturée).	Offres	—

N°	Règle	Service	Test
RG-14	Une offre est disponible si elle est active, que sa date limite n'est pas dépassée et qu'il reste au moins une place.	Offres	ST-11
RG-15	Compatibilité : la filière de l'étudiant doit être ciblée (liste vide = toutes), son niveau au moins égal au niveau minimum et la date limite non dépassée ; le score de compétences est indicatif.	Offres	ST-10
RG-16	L'acceptation d'une candidature réserve une place ; à zéro place, l'offre passe à `POURVUE`. La réservation ne peut pas rendre le compteur négatif.	Offres, BPMN	PT-01
Candidatures			
RG-17	Un étudiant ne peut avoir qu'une candidature non annulée par offre.	Candidatures, BPMN	RT-02, PT-02
RG-18	Une candidature n'est enregistrée que si toutes les vérifications du processus sont satisfaites ; sinon l'étudiant reçoit un rejet motivé.	BPMN	PT-02
RG-19	Les transitions de statut suivent l'automate de la figure 5.1 ; tout changement est historisé (auteur, date, motif).	Candidatures	RT-03
RG-20	Une candidature ne peut être annulée que tant qu'elle est `DEPOSEE` (l'entreprise ne l'a pas encore prise en charge), par l'étudiant concerné ou le responsable ; l'annulation interrompt le processus.	Candidatures, BPMN	RT-03, PT-07, PT-09
RG-31	Un étudiant ne peut avoir plus de 5 candidatures en cours (déposées ou en évaluation) ; au-delà, la candidature est rejetée.	Candidatures, BPMN	PT-10
RG-21	La décision sur une candidature orchestrée passe obligatoirement par la tâche « Évaluer » du processus.	Candidatures	—
RG-22	Seule l'entreprise qui a publié l'offre (ou le responsable) peut rendre la décision.	Processus	PT-01
RG-23	Une demande de complément exige un commentaire, transmis à l'étudiant ; celui-ci peut alors compléter son dossier.	Processus	PT-03
Notifications			
RG-24	L'étudiant est notifié par e-mail du dépôt, du rejet, d'une demande de complément, de la décision et d'un éventuel abandon ; l'entreprise de la réception et des relances.	Notifications, BPMN	PT-01, PT-03
RG-25	Sans décision après le délai fixé (3 jours par défaut), l'entreprise est relancée sans que l'évaluation soit annulée.	BPMN	PT-08
RG-26	Les étudiants actifs au profil compatible reçoivent un rappel unique par offre clôturant dans les 3 jours.	Notifications	—

N°	Règle	Service	Test
RG-27	Une notification non envoyée ne bloque jamais le processus ; l'incident est consigné.	BPMN	PT-06
Stages			
RG-28	Un seul stage par candidature acceptée ; la date de fin est postérieure à la date de début ; le stage est `PLANIFIE` si son début est futur, `EN_COURS` sinon.	Stages	RT-05
RG-29	Les comptes rendus ne sont acceptés que pour un stage planifié ou en cours ; la note d'une évaluation est comprise entre 0 et 20 et il n'y a qu'une évaluation finale.	Stages	RT-05
RG-30	L'évaluation finale clôture le stage ; à la fin ou à l'interruption du stage, l'étudiant redevient `ACTIF`.	Stages	RT-05

CHAPITRE 9

Scénarios de test

9.1 Stratégie

Trois niveaux de tests complémentaires ont été mis en place :

1. **Tests d'intégration automatisés** (JUnit 5, `PlateformeIT`) : la plateforme complète est démarrée sur des bases H2 en mémoire puis sollicitée par de *vrais* appels SOAP (clients JAX-WS générés depuis le WSDL) et HTTP. Les tests couvrent les services un par un puis le processus de bout en bout, y compris les pannes. Ils s'exécutent à chaque `mvn package` et ont également été exécutés sur **PostgreSQL 16**.
2. **Collection Postman** (`docs/postman`) : 39 requêtes enchaînées avec 45 assertions sur les codes et les contenus, authentifiées par clé d'API, exécutable dans le *Collection Runner* ou avec Newman.
3. **Requêtes SoapUI** (`docs/soap`) : enveloppes prêtes à l'emploi pour chaque type d'opération, accompagnées des réponses réellement obtenues.

9.2 Tests automatisés

TABLE 9.1 – Scénarios de test automatisés et résultats

ID	Scénario	Résultat attendu	Résultat
Services SOAP			
ST-01	Récupérer les trois WSDL	Opérations présentes dans chaque contrat	**OK**
ST-02	Appel SOAP sans en-tête WS-Security	Fault `FailedAuthentication`	**OK**
ST-03	Ajouter, consulter, modifier, supprimer un étudiant	Statut `ACTIF` par défaut ; consultation après suppression : `RESSOURCE_INTROUVABLE`	**OK**
ST-04	Ajouter un étudiant (matricule, e-mail et niveau invalides)	Fault `DONNEES_INVALIDES` avec 3 détails	**OK**
ST-05	Ajouter un matricule existant	Fault `CONFLIT`	**OK**
ST-06	Un étudiant consulte une autre fiche / supprime sa fiche	Fault `ACCES_REFUSE`	**OK**
ST-07	Vérifier un étudiant actif, suspendu, inexistant	éligible / non éligible / inexistant	**OK**
ST-08	CRUD entreprise ; offres d'une entreprise ; suppression avec offres actives	2 offres ; `REGLE_METIER`	**OK**
ST-09	Publier une offre : entreprise non référencée, autre entreprise, entreprise propriétaire	`REGLE_METIER`, `ACCES_REFUSE`, offre `ACTIVE`	**OK**

ID	Scénario	Résultat attendu	Résultat
ST-10	Compatibilité AMOA/M2, DATA/L3 et offre expirée	compatible, score 50 % ; incompatible, 2 motifs ; incompatible (date limite)	**OK**
ST-11	Disponibilité d'une offre expirée	date dépassée, non disponible	**OK**
ST-12	Fiche d'un candidat consultée par une entreprise puis par le responsable	téléphone et e-mail masqués, adresse absente ; fiche complète	**OK**
Services REST			
RT-01	Requêtes sans identifiants, avec mauvais mot de passe, rôle étudiant sur `POST /stages`	401, 401, 403	**OK**
RT-02	Dépôt invalide, valide, en double, JSON mal formé	400, 201 `DEPOSEE`, 409, 400	**OK**
RT-03	Transition interdite, annulation par un tiers, passage en évaluation, annulation refusée, historique ; annulation d'une candidature déposée	409, 403, 200, 422, 3 entrées ; `ANNULEE` puis 422	**OK**
RT-04	Notification d'acceptation ; lecture par un autre étudiant ; type inconnu	201 et message rendu ; 403 ; 400	**OK**
RT-05	Stage : création, doublon, compte rendu, note 25, évaluation finale, historique	201 `EN_COURS`, 409, 201, 400, `TERMINE`, moyenne 16,5, étudiant `ACTIF`	**OK**
RT-06	Clé d'API valide, invalide, régénérée ; ancienne clé	200 ; 401 `CLE_API_INVALIDE` ; 200 ; 401 (révoquée)	**OK**
Processus BPMN			
PT-01	Scénario nominal : candidature puis acceptation par l'entreprise	Stage créé aux dates saisies, étudiant `EN_STAGE`, offre `POURVUE`, 3 notifications ; tâche invisible pour une autre entreprise	**OK**
PT-02	Candidatures non recevables : étudiant suspendu, niveau insuffisant, date dépassée, entreprise non référencée, offre pourvue	5 instances terminées par « Candidature rejetée » avec motif	**OK**
PT-03	Complément sans commentaire, complément, puis refus	400 ; retour à l'évaluation ; `REFUSEE` ; notifications `DEMANDE_COMPLEMENT` et `REFUS`	**OK**
PT-04	Service Candidatures en panne pendant les vérifications, puis relance	`INCIDENT_TECHNIQUE` (étape `VERIFICATIONS`) puis `EN_ATTENTE_DECISION`	**OK**
PT-05	Service Stages en panne pendant la finalisation, puis abandon	`INCIDENT_TECHNIQUE` (étape `FINALISATION`) puis `ABANDONNEE`	**OK**
PT-06	Service Notifications en panne pendant le dépôt	Processus poursuivi, incident consigné	**OK**
PT-07	Annulation d'une candidature déposée	Instance `INTERROMPU`	**OK**
PT-08	Délai de décision dépassé (2 s en test)	Relance de l'entreprise, évaluation toujours en attente	**OK**

ID	Scénario	Résultat attendu	Résultat
PT-09	Prise en charge par l'entreprise puis tentative d'annulation	`DEPOSEE` → `EN_COURS_EVALUATION`, tâche attribuée, annulation refusée (422)	**OK**
PT-10	Sixième candidature en cours pour un même étudiant	Rejet motivé par le processus ; 422 sur l'API	**OK**

Listing 9.1 – Exécution de la suite (extrait de la sortie Maven Surefire)

```
$ mvn test
Tests run: 12, Failures: 0, Errors: 0, Skipped: 0 -- in PlateformeIT$Soap
Tests run: 6,  Failures: 0, Errors: 0, Skipped: 0 -- in PlateformeIT$Rest
Tests run: 10, Failures: 0, Errors: 0, Skipped: 0 -- in PlateformeIT$Processus

$ mvn test -Ddb.url=jdbc:postgresql://localhost:5433/stages -Dbpmn.db.url=jdbc:postgresql://...
Tests run: 24, Failures: 0, Errors: 0, Skipped: 0
```

Listing 9.2 – Exemple de test : indisponibilité puis relance (PT-04)

```
@Test
@DisplayName("PT-04 Indisponibilité d'un service : incident, tâche du responsable, relance ré
    ussie")
void incidentRelance() {
    SimulateurPannes.definir("candidatures", true);
    JsonNode r;
    try {
        r = soumettre("E2026003", 4).json();
    } finally {
        SimulateurPannes.definir("candidatures", false);
    }
    assertEquals("INCIDENT_TECHNIQUE", r.get("statut").asText());
    assertEquals("VERIFICATIONS", r.get("variables").get("etapeCourante").asText());
    JsonNode apres = Http.post("/api/processus/taches/" + tache(r, "traiterIncident") + "/
    incident",
            ADMIN, ADMIN_PWD, Map.of("action", "RELANCER")).json();
    assertEquals("EN_ATTENTE_DECISION", apres.get("statut").asText());
}
```

9.3 Collection Postman

La collection est organisée en six dossiers (système, processus, candidatures, notifications, stages, pannes). Les variables de collection (`tacheId`, `candidatureId`, `stageId`, `incidentId`) sont alimentées par les scripts de test, ce qui permet d'enchaîner un scénario complet.

Listing 9.3 – Exécution de la collection avec Newman

```
$ newman run docs/postman/gestion-stages.postman_collection.json
|                 requests |                 39 |                 0 |
|               assertions |                 45 |                 0 |
```

9.4 Requêtes SoapUI

Après import des trois WSDL dans SoapUI, les enveloppes de `docs/soap/` peuvent être collées telles quelles :

Table 9.2 – Requêtes SOAP fournies

Fichier	Cas	Résultat obtenu
`01-etudiants-ajouterEtudiant`	Création d'une étudiante	fiche `ACTIF`
`02-etudiants-consulterEtudiant`	Consultation	fiche complète
`03-etudiants-verifierEtudiant`	Étudiante suspendue	`eligible=false`
`04-etudiants-ajouterEtudiant-invalide`	Données invalides	Fault, 3 détails
`05-entreprises-verifierEntreprise`	Entreprise non référencée	`referencee=false`
`06-entreprises-consulterOffresEntreprise`	Composition SOAP	2 offres
`07-offres-creerOffre`	Publication d'une offre	offre `ACTIVE`
`08-offres-verifierDisponibilite`	Offre expirée	`disponible=false`
`09-offres-verifierCompatibilite`	Score de compétences	50 %, 2 manques
`10-sans-ws-security`	Sans authentification	`wsse:FailedAuthentication`

CHAPITRE 10

Interface graphique (bonus)

10.1 Principes

L'interface est une application web monopage (HTML, CSS, JavaScript sans framework) servie par la plateforme. Elle consomme *directement* les services : un petit client SOAP en JavaScript (`soap.js`) construit les enveloppes avec l'en-tête WS-Security et convertit les réponses XML, tandis que les services REST sont appelés en JSON. Le diagramme BPMN est affiché avec **bpmn-js**, embarqué dans l'application, ce qui permet de suivre une instance en direct. La charte reprend le bleu de l'INPT. Après connexion, le menu s'adapte au rôle :

TABLE 10.1 – Écrans par acteur

Acteur	**Écrans**
Étudiant	Offres disponibles avec score d'adéquation et motifs d'incompatibilité ; dépôt de candidature (déclenche le processus) ; suivi des candidatures, historique, complément et annulation ; notifications ; « Mon stage » (progression, comptes rendus, évaluations)
Entreprise	Candidatures à évaluer (profil, compétences, lettre, CV) et formulaire de décision ; publication et clôture d'offres (SOAP) ; candidatures reçues ; stagiaires, démarrage du stage et évaluations
Responsable	Tableau de bord ; processus BPMN et instances (diagramme surligné, trace, variables) ; incidents (relancer, abandonner) ; gestion des étudiants et entreprises (SOAP) ; offres, candidatures et stages ; supervision (pannes simulées, campagne de rappels)

10.2 Parcours de l'étudiant

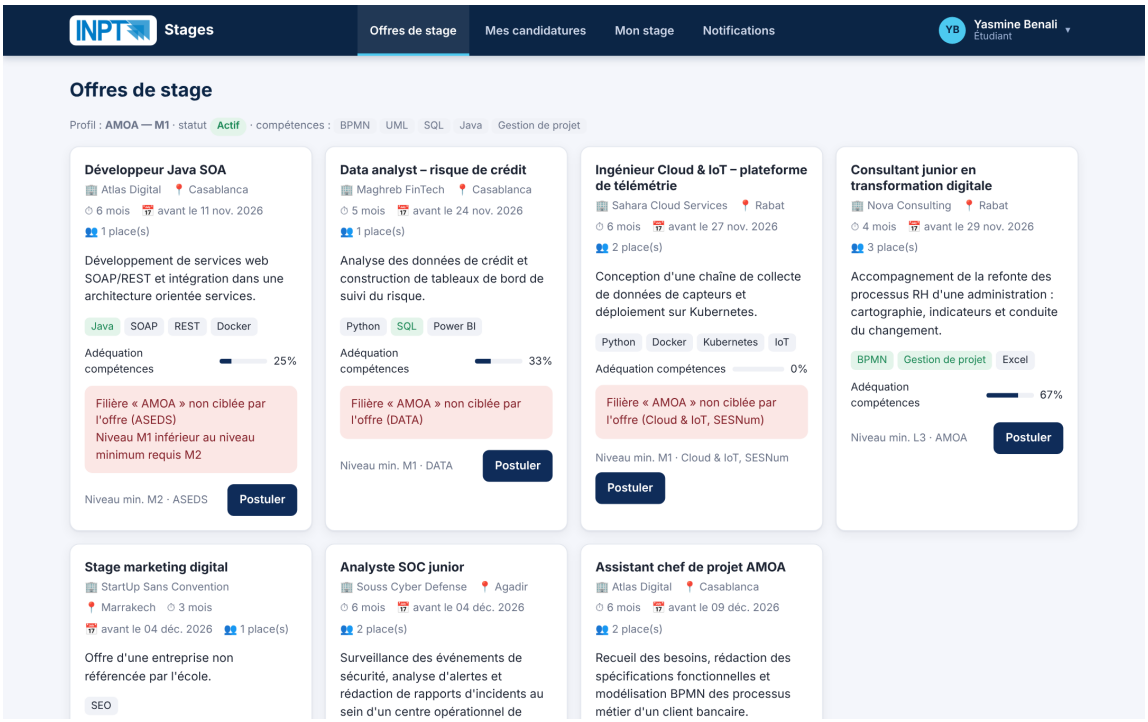
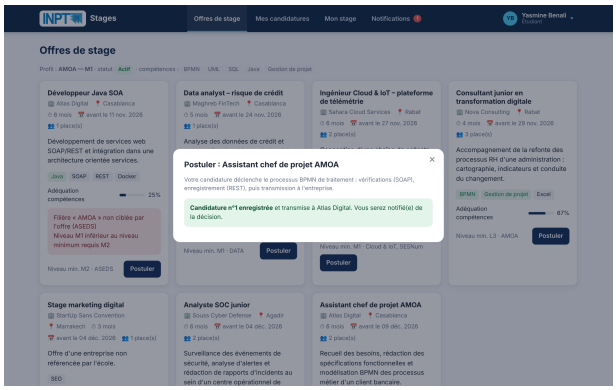
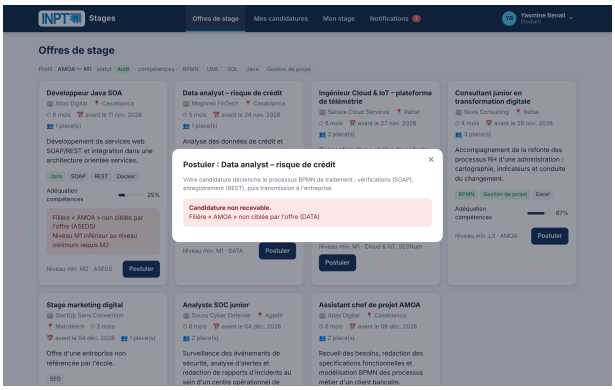


FIGURE 10.1 – Offres de stage : score d'adéquation et motifs d'incompatibilité calculés par `verifierCompatibilite`

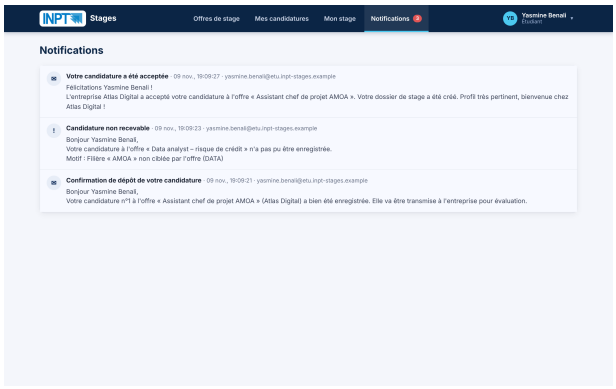


(a) candidature enregistrée



(b) rejet motivé par le processus

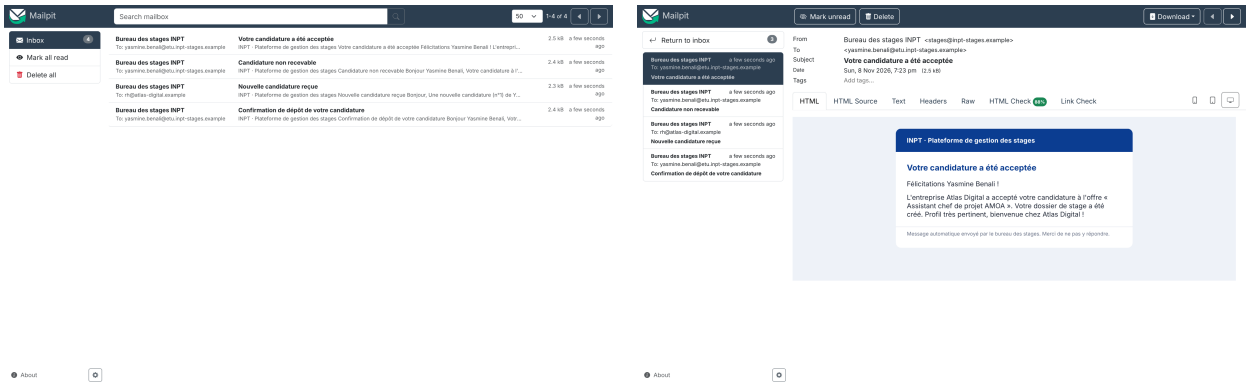
FIGURE 10.2 – Résultat d'une soumission de candidature



(a) notifications reçues (b) suivi du stage

FIGURE 10.3 – Notifications et suivi du stage

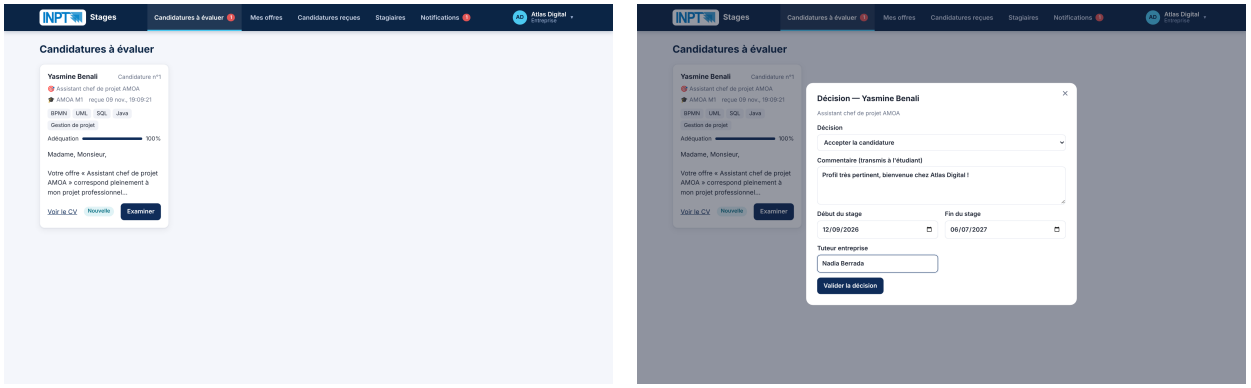
10.3 E-mails reçus



(a) boîte de réception Mailpit (b) e-mail d'acceptation (HTML)

FIGURE 10.4 – Vrais e-mails envoyés en SMTP par le service Notifications et capturés par Mailpit

10.4 Parcours de l'entreprise



(a) candidatures à évaluer (b) décision (tâche humaine BPMN)

FIGURE 10.5 – Évaluation d'une candidature par l'entreprise : « Examiner » prend la candidature en charge, puis la décision est rendue

10.5 Parcours du responsable

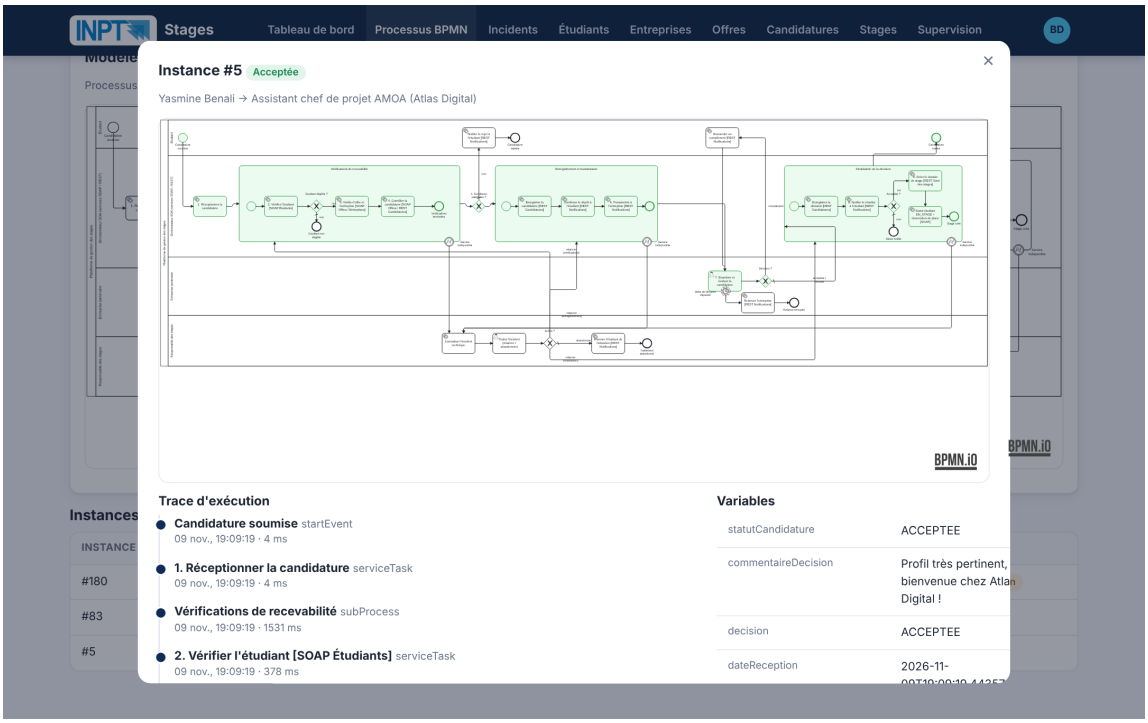
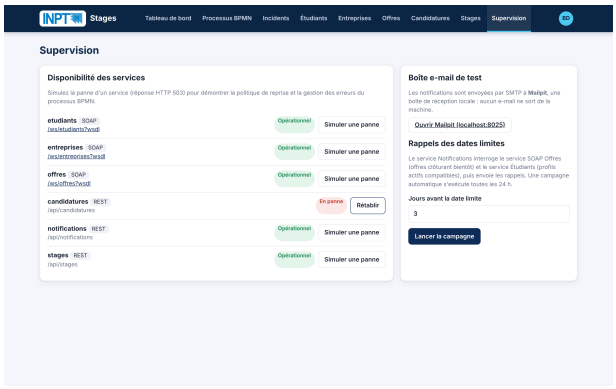
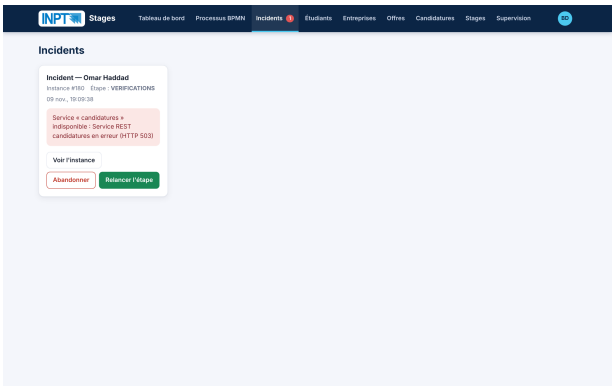


FIGURE 10.6 – Suivi d'une instance : activités traversées en vert sur le diagramme, trace d'exécution et variables

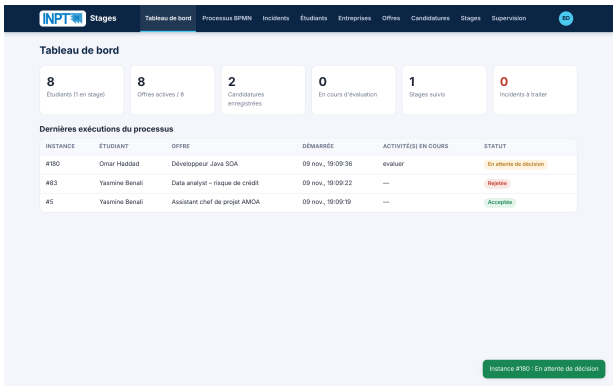


(a) simulation d'une panne

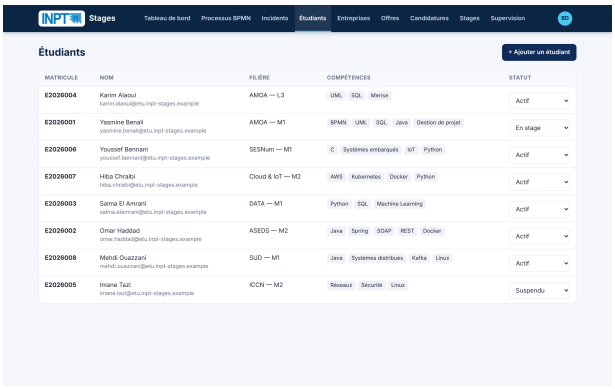


(b) incident à traiter

FIGURE 10.7 – Gestion d'une indisponibilité de service depuis l'interface



(a) tableau de bord



(b) gestion des étudiants (SOAP)

FIGURE 10.8 – Écrans d'administration

CHAPITRE 11

Bilan et perspectives

11.1 Bilan

Le projet livre une plateforme complète et opérationnelle : six services autonomes aux contrats explicites, trois en SOAP décrits par WSDL et trois en REST, orchestrés par un processus BPMN 2.0 réellement exécuté. Le processus couvre les neuf étapes demandées et va au-delà sur trois points : demande de complément d'information, relance automatique de l'entreprise et interruption sur annulation. La gestion des erreurs combine reprise automatique, événements d'erreur BPMN et traitement humain des incidents, avec des reprises idempotentes. L'ensemble est vérifié par 28 tests d'intégration de bout en bout, sur H2 et PostgreSQL, et peut être utilisé par chaque acteur à travers l'interface web.

11.2 Limites

— Les six services partagent un processus et une base physique pour simplifier la démonstration ; l'isolation est logique (tables, contrats, appels réseau), pas physique.

— Le transport n'est pas chiffré dans l'environnement de démonstration (HTTPS à activer en production).

11.3 Perspectives

— Déployer chaque service dans son propre conteneur avec sa base (Docker Compose ou Kubernetes) et un annuaire de services ou une passerelle d'API.

— Remplacer HTTP Basic par des jetons OAuth 2 / JWT, et signer les messages SOAP (WS-Security *Signature*).

— Ajouter un disjoncteur (*circuit breaker*) et une file de messages pour les notifications.

— Remplacer Mailpit par le serveur SMTP de l'école et ajouter des notifications instantanées dans l'application.

— Enrichir la compatibilité par une recommandation d'offres fondée sur l'historique des stages.

ANNEXE A

Mise en œuvre

A.1 Lancement

```
mailpit                              # boîte e-mail de test : http://localhost:8025 (SMTP 1025)
mvn package                          # compile, teste (28 tests) et produit le jar
java -jar target/gestion-stages.jar
# Interface : http://localhost:8090/       WSDL : http://localhost:8090/ws/etudiants?wsdl

docker compose up --build            # variante tout-en-un : plateforme + PostgreSQL + Mailpit
cd rapport && latexmk -pdf rapport.tex
```

A.2 Comptes de démonstration

TABLE A.1 – Comptes de démonstration

Identifiant	Mot de passe	Profil
admin	Admin@2026	Responsable des stages
E2026001	Etudiant@2026	Yasmine Benali — AMOA, M1 (2A)
E2026002	Etudiant@2026	Omar Haddad — ASEDS, M2 (3A)
E2026003	Etudiant@2026	Salma El Amrani — DATA, M1
E2026004	Etudiant@2026	Karim Alaoui — AMOA, L3 (1A)
E2026005	Etudiant@2026	Imane Tazi — ICCN, M2, *suspendue*
E2026006 à E2026008	Etudiant@2026	Youssef Bennani (SESNum), Hiba Chraibi (Cloud & IoT), Mehdi Ouazzani (SUD)
atlas, nova, fintech, cyber, cloud	Entreprise@2026	Entreprises référencées
startup	Entreprise@2026	Entreprise non référencée
systeme	Systeme@2026	Compte technique de l'orchestrateur

Chaque compte dispose d'une clé d'API de démonstration `stg_demo_<identifiant>` (par exemple `stg_demo_admin`), utilisable dans l'en-tête `X-API-Key` des appels REST.

A.3 Organisation du code source

```
src/main/java/com/universite/stages/
  soap/etudiants|entreprises|offres     services 1 à 3 : contrat, implémentation, DAO, DTO JAXB
  rest/candidatures|notifications|stages services 4 à 6 : ressources JAX-RS, DAO
  rest/processus                        API d'orchestration
  bpmn/                                 moteur Flowable et délégués des tâches de service
  clients/                              clients SOAP dynamiques, client REST, politique de
    reprise
  security/                             WS-Security, PBKDF2, rôles
  common/                               configuration, JDBC, validation, simulateur de pannes
src/main/resources/
  processes/candidature-stage.bpmn20.xml  processus BPMN 2.0 (modèle + diagramme)
```

```
  schema.sql  application.properties  public/ (interface web)
src/test/java/.../PlateformeIT.java     24 tests d'intégration
docs/  wsdl/  soap/  postman/  bonita/GUIDE_BONITA.md  SCENARIO_VIDEO.md
tools/ generer_bpmn.py  rendre_bpmn.js  captures_ui.js
```

A.4 Vidéo de démonstration

Le déroulé proposé pour la vidéo (environ 10 minutes) est fourni dans `docs/SCENARIO_VIDEO.md` : tests SOAP dans SoapUI, collection Postman, parcours étudiant, décision de l'entreprise, simulation de panne et relance par le responsable, puis exécution des tests automatisés.